Maximum Sub-array Sum

Taylor Accardi
California State Polytechnic University, Pomona
Pomona
tjaccardi@cpp.edu

Feng Ru Yang
California State Polytechnic University, Pomona
Pomona
fyang@cpp.edu

Matthew Puente
California State Polytechnic University, Pomona
Pomona
mtpuente@cpp.edu

***Abstract* - Maximum Sub-array Sum problem has many applications in biology and computer science. The algorithm works to find the greatest sum in a continuous subset which can assist in identifying patterns. In this paper, three methods will be proposed to solve the problem along with the analysis of the time complexity for each solution.**

I. INTRODUCTION

The Maximum Sub-array Sum was proposed in 1977 by Ulf Grenander as a solution to the maximum likelihood estimation method in statistics with computers. Shortly thereafter multiple solutions were proposed using a two dimensional array starting with a time complexity of O(n6). To better analyze the structure, the problem was analyzed as a one dimensional array. First, Grenander develop a O(n2) solution. Then,Michael Shamos using divide-and-conquer, a solution was found with a time complexity of O(n log n). Following this development a solution was found in O(n) time which is clearly the best possible case.

The problem is to find the maximum sum of a continuous sub-array in a given array. For example, array [-2, -5, 6, -2, -3, 1, 5, -6] to which the solution would be [6, -2, -3, 1, 5]. Not only this problem was intended for estimating patterns in images, but this information can also be useful in fields such as computational biology to analyze genome sequences and computer vision for detecting brightness.

Many algorithms could be applied to solve the problem. In this research paper we will focus on how to solve the problem first by the brute force approach, then use Kadane's algorithm to improve it's time complexity. There are two brute force approaches, one uses three for-loops, while the other is the improvement of that which only requires two for-loops. This report will mainly focus on the improved brute force approach, that yield the time complexity of O(n2). To further minimize the run time, dynamic programming offers a better solution. This solution is the Kadane's Algorithm with the time complexity of O(n). There are other algorithms that could solve this problem; however, this paper will only focus on these two approaches.

Without using techniques derived from dynamic programming the problem quickly expands out of control with many sub arrays formed from the problem array. While using dynamic programming quickly decreases the time complexity at the cost of marginally more space. In this paper we cover trivial and technique derived methods with time complexities of O(n2) and O(n) demonstrating the effectiveness of dynamic programming.

II. ALGORITHM DESCRIPTIONS

In the following sections we will focus on the two brute force approach and the Kadanes' algorithm which is a dynamic approach to solve the maximum subarray problem. The following three algorithms make up three out of the greater pool of solutions. The first algorithm will use a brute force approach to solve the problem, the second algorithm is a small improvement over the first brute force algorithm and the third is build on top of a solution utilizing dynamic programming while incurring a small memory footprint on the system.

A) Brute Force Approach one

Use a variable to calculate all the possible contiguous sums in the array. Then have another variable to keep the largest sum, and update the largest sum variable by making comparisons. When all the sums are compared, the result is in the largest sum variable.

\

```
public int worseBruteForceGetMaxSub() // uses the int array to get
maximum sub array in worst possible way
 {
 int largestSum = Integer.MIN_VALUE; // initialize largest value

    for(int i=0;i<numbers.length;i++) //first index loop
    {
      for(int j=i; j<numbers.length;j++) //second index loop
      {
            int currentSum = 0; //current sum value
            for(int k =i; k<=j; k++) // loop to find current sum
                  currentSum += numbers[k];
     largestSum = Math.max(largestSum, currentSum);
     // compare between current and largest sum
      }
    }
       return largestSum; // return largest sum
}
```

Figure i: Brute force algorithm 1

B) *Brute Force Approach two*

This is the improved version of the previous approach. It avoids recalculating the current sum by only reinitializing the current sum when updating the outer loop. In other words the current sum in the inner loop is calculated by adding the new value to the existing current sum.

```
public int bruteForceGetMaxSub() // Second worst way to find largest
sum
      {
            int largestSum = Integer.MIN_VALUE; // initialize largest
sum
            int currentSum; // initialize current sum
            for(int i =0;i<numbers.length;i++) // first index counter
            {
                  currentSum = 0; // reset current sum
                  for(int j=i; j<numbers.length;j++) // second index
counter
                  {
                  currentSum += numbers[j]; // add most recent value
to current sum
                  largestSum = Math.max(largestSum, currentSum);
 // compare between largest and current sum
                  }
          }

            return largestSum; // return largest sum
       }
```

Figure ii: Brute force algorithm 2

C) *Dynamic Programming Approach one*

This algorithm is one of the most efficient algorithms. It achieves that by copying down the original array into a temporary array. It then goes through the array once, and at each index in the array it sees what the largest sum ending in that index would be. In order to do this it compares the largest sum of the previous element added to the value in the current index with the value in the current index, and sees which one is larger. This value is the largest sum that can end at the current index in the array. We then compare that sum with the largest sum we've seen so far and see which is larger. It repeats this process until it reaches the end of the array, at which point the largest value is saved in the largest sum variable.

D) *Dynamic Programming Approach two*

Kadane's algorithm works in the same way as the other dynamic approach algorithm, however, it has a better space complexity. While the original dynamic approach copies down the original array, Kadane's only tracks indices in the array. This allows the algorithm to compute the maximum subarray sum in 0(n) time with 0(1) space complexity because it never actually copies down the array.

```
public int getKadanesMaxSub() // Best way to find largest sum
    {
    int currentMax = 0; // initialize current sum
    int totalMax = 0; // initialize final sum
    for(int i =0;i<numbers.length;i++) // loop through array
    {
    currentMax = Math.max(currentMax +numbers[i], numbers[i]);
 //compare between last max+the new value, and the new value
   totalMax = Math.max(totalMax, currentMax);
// compare between total max and current max
    }
     return totalMax; // return current max
}
```

Figure iii: Dynamic programming solution

III. TIME COMPLEXITY

The time complexity of these three algorithms falls well within the manageable range only feasibly stressed due to the input size given.Unlike divide-and Conquer approach, where time complexity has to be solve using recurrence relation and Master Theorem, both the two brute force and kadanes' algorithm have a very straightforward representation of the time complexity - for loops.

A) *Brute Force Approach one*

As shown in the graph, the actual time required to run the program is n to the power of three. Our experimental values match the theoretical hypothesis of $O(n^3)$. The line of best fit matches almost perfectly to the actual trendline of the time vs. array size graph.

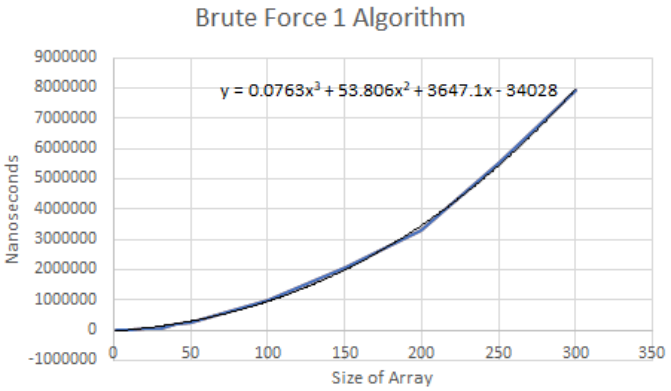


Figure iv: Brute force algorithm 1 time complexity graphed

B) Brute Force Approach two

While not a large improvement over the first algorithm it represents progress. By taking out the extra for-loop the time complexity has been reduced by a magnitude of n to O(n2). Still though, the time complexity is quite inefficient and could be improved with other programming methodologies.In this graph we see the trendline of the actual graph is slightly different than the predicted n2 equation most likely due to other factors outside of the algorithm its self.

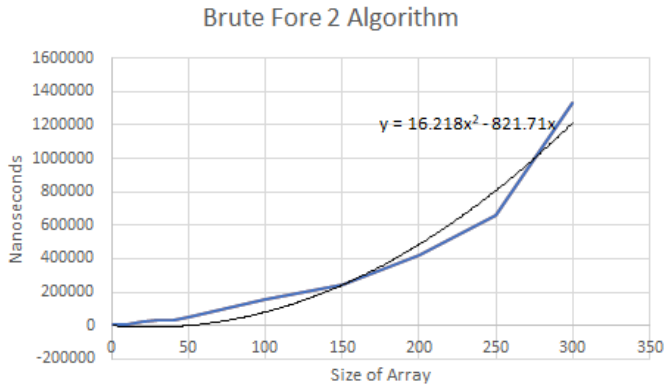
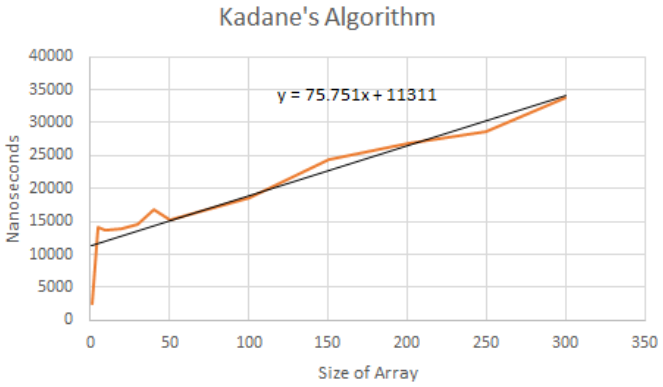


Figure v: Brute force algorithm 2 time complexity graphed

C) Dynamic Approach (Kadane's Algorithm)

By utilizing the extra methodologies from dynamic programming we can reduce the time complexity to its best of yet form making it linear. At this point the time complexity is directly from the input size as the programming has been reduced to a single for loop. Comparing the graph of Kadanes' algorithm, to the other two brute force methods, we see that initially when the array size is smaller the time it took for each algorithm to complete the task is not different by much. However, when the size increases the time required to complete the task differs drastically.



IV. ADVANTAGES AND DISADVANTAGES

Each approach to the problem ended up becoming an iteration of the last. During the process of using and testing each approach we noticed distinct advantages and disadvantages.

A) Brute Force

For the sake of our two brute force attempts being so similar we decided to treat them as one when we considered this approach as opposed to our dynamic programming approach.

Advantages:

Since the time complexity of the brute force approaches are not ideal, the advantage of this approach is it's simplicity for understanding the algorithm. The concept behind this approach is very straightforward as its name suggests.

Disadvantages:

On the other hand, this approach does not offer any advantage on time complexity. Compared to Kadanes' algorithm with time complexity O(n), the brute force is that of the square of Kadanes' time complexity on it's improved version.

B) Kadanes:

The dynamic programming approach, because it utilized a different fundamental design, is seperate and thus is considered on its own.

Advantages:

Kadanes' algorithm offers one of the best time complexity for the maximum subarray problem with

time complexity of O(n). In addition, this algorithm was designed on top of the other dynamic approach that uses an array to hold the cumulative sum. Therefore, despite it having the time complexity of O(n), it's space complexity is O(n). The Kadanes' algorithm adjusts the problem by storing the sum in the original array that reduces it's space complexity to O(1).

Disadvantages:

There is no significant disadvantage for Kadane's algorithm. Being the algorithm of the best time complexity and space complexity, the algorithm outperforms all others. The concept can be difficult to explain but in general becomes simple with insight.

V. RELATED WORK

A) Subset sum problem:

This problem is about finding a nonempty set of elements in an array where the sum of the element in the set is equal to zero. While it is easy to confirm the solution, checking whether the solution exists is a challenge. Similar to this problem of finding the maximum subarray sum, dynamic approach gives the problem a better option for its time complexity [3]. By doing so, it reduce the time complexity from exponential to polynomial.

B) Maximum Subarray Problem (Two-Dimensional Array):

This problem is originally a 2-D mentional array problem. In that case, the Kadanes' algorithm does not apply, but it can be used in the case when the array becomes one dimensional. In the paper by Takaoka, their team uses the Distance Matrix Multiplication process to reduce the time complexity from cubic to O(m2n(log logm /log m1/2)) [1]. However, this is the worst time complexity, in reality the algorithm will be expected to run faster. This is part of an All Pairs Shortest Path Problem, any improvement in the algorithm solving such problem would accelerate the time to solve this problem.

C) K-th Maximum Sums Problem:

This is the extension of the maximum subarray sum problem, where in this case, we are trying to find the kth largest sum in an array. An efficient approach for this problem is using dynamic approach as well, but instead of an array data structure, min heap or priority queue is used to implement the code [5]. First, we find the continuous sums, then sort them using a min heap until k elements are reached. Second, we make a min heap by comparing each element with the kth element. If the element is greater, then we pop out the top of the heap (min) and insert the new element. At the end, the top element of the heap will be the result. This approach will yield a O(n2log k) time complexity.

VI. TEAM FINDINGS

The best algorithm developed is the kadane algorithm which can find the maximum subarray sum in O(n) time. This algorithm utilizes trivial forms of dynamic programming such that it 'memorizes' the best result so far. In doing so the algorithm only needs to iterate over n elements thus giving us our O(n) result. In a two-dimensional array this time complexity then becomes O(n+k) where n is our rows and k is our columns. This is a huge improvement over the original O(n6) time initially devised in 1977 for a two dimensional array and then later in O(n3) with a one-dimensional implementation.

With an algorithm that runs in O(n) time this is almost certainly the best case scenario disregarding any case where the time complexity is less than n.

VIII. CONCLUSION

In the duration of our work we found that absolutely Kadan's algorithm is the best case scenario for the problem at hand. The two dimensional array of course adds an extra layer on to the level of complexity that still limits by input size but is by no means inefficient.

While we worked through the various approaches to the Maximum Subarray Sum problem we found increasingly that through clever use of Dynamic Programming methodologies the time complexity saw massive improvements going straight from squared to linear time complexity. Other approaches that we thought up but did not pursue included utilizing statistics such as means and median though we repeatedly came upon the issue that the correct solution could not be determined with accuracy.

Throughout the course of the project we encountered challenges though curiously we found a lack of documentation on the use of such an algorithm. In addition, since Kadanes' algorithm is in O(n) time complexity, further research on this topic seems to be stopped. Beside, once the underlying concept was understood Kadanes' algorithm offers the best solution among the others.

In conclusion, in this paper we summarize the different approaches to solve the maximum subarray problem. We then focus on the brute force and the dynamic approach by analyzing it's time complexity and the application. Overall, we came to the conclusion that Kadanes' algorithm offers us the best solution both in terms of the time constraint and the space constraint.

IX. REFERENCES

[1] Takaoka, T., 2020. Efficient Algorithms For The Maximum Subarray Problem By Distance Matrix Multiplication. ScienceDirect.

[2] Singhal, R., 2020. Kadane'S Algorithm—(Dynamic Programming)—How And Why Does It Work?. [online] Medium. Available at: <https://medium.com/@rsinghal757/kadanes-algorithm-dynamic-programming-how-and-why-does-it-work-3fd8849ed73d> [Accessed 12 April 2020].

[3] Afteracademy.com. 2020. Maximum Subarray Sum. [online] Available at: <https://afteracademy.com/blog/maximum-subarray-sum> [Accessed 21 April 2020].

[4] GeeksforGeeks. 2020. Dynamic Programming - Subset Sum Problem. [online] Available at: <https://www.geeksforgeeks.org/subset-sum-problem-dp-25/> [Accessed 13 April 2020].

[5] Brodal, G. and Jørgensen, A., 2007. Mathematical Foundations Of Computer Science 2007. Berlin: Springer.